Com S 228 Recitation - Week 3
Access Modifier, Exceptions, Deep Copying, etc.

Recitation 7

Iowa State University

February 4, 2019

Announcements

- Project 1 is due February 9.
- Exam 1 is Wednesday, February 20.

Table of Contents

Usage of this and super

Usage of `this` keyword

`this` **refers to a current object.**

When to use?

1. **Disambiguate variable references from arguments.**

```
public class Foo
{
    private String name;

    public void setName(String name) {
        this.name = name;
    }
}
```

Usage of `this` keyword

`this` **refers to a current object.**

2. **Using** `this` **as an argument passed to another object.**

```
public class Foo {
    public String useBarMethod() {
        Bar theBar = new Bar();
        return theBar.barMethod(this);
    }
    public String getName() {
        return "Foo";
    }
}

public class Bar {
    public void barMethod(Foo obj) {
        obj.getName();
    }
}
```

Usage of `this` keyword

`this` **refers to a current object.**

3. **Using** `this` **to call alternate constructors**,
   when you have multiple constructors for a single class.

```
class Foo {
    public Foo() {
        this("Some default value for bar");

        //optional other lines
    }

    public Foo(String bar) {
        // Do something with bar
    }
}
```

Usage of `super` keyword

If your method overrides one of its superclass's methods, you can invoke the overridden method through the use of the keyword super.
You can also use `super` to refer to a hidden field *(although hiding fields is discouraged)*.

1. Invoke a superclass's constructor
   - `super()`
   - `super(parameter list)`
2. Accessing superclass members
   - `super.superClassVariable;`
   - `super.superClassMethod();`

[0] Java-doc: Using the Keyword super

Usage of super keyword

`super.varName` **accesses the parent class variable.**

```
class Parentclass {
    int num = 100;
}

class Subclass extends Parentclass {
    int num = 80;
    void printNumber(){
        // Super.variable_name
        System.out.println(super.num);
    }
}
```

Usage of super keyword

`super()` **calls the parent constructor with no arguments.**

```
class Parentclass
{
    Parentclass(){ System.out.println("Superclass");}
}
class Subclass extends Parentclass
{
    Subclass(){
        /* implicitly inserts a no-argument super() */
        System.out.println("Constructor of Subclass");
    }
    Subclass(int num) {
        /* implicitly inserts a no-argument super() */
        System.out.println("Constructor with arg");
    }
}
```

Usage of super keyword

`super(oaram list)` **calls the parent constructor with arguments.**

```
class Parentclass {
    /* Parent class has no default constructor */
    Parentclass(int num){
        System.out.println("Superclass:" + num);
    }
}
class Subclass extends Parentclass {
    Subclass(){
        super(5); // must be explicitly added to the first line
        System.out.println("Constructor of Subclass");
    }
    Subclass(int num) {
        super(5); // must be explicitly added to the first line
        System.out.println("Constructor with arg");
    }
}
```

Exception handling

Exception

```
readFile {
    open the file;
    determine its size;
    allocate that much memory;
    read the file into memory;
    close the file;
}
```

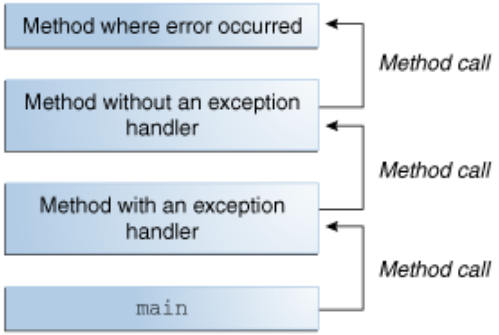
Potential errors:

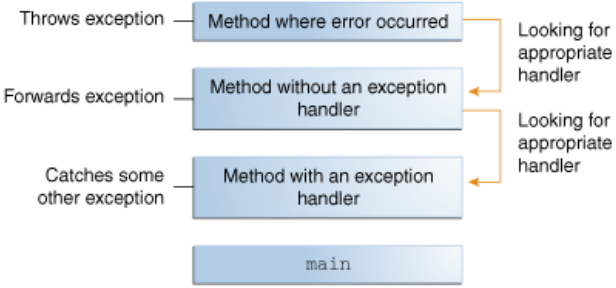
- What happens if the file can't be opened?
- What happens if the length of the file can't be determined?
- What happens if enough memory can't be allocated?
- What happens if the read fails?
- What happens if the file can't be closed?

Exception handling

Exceptions

An exception is an event, which occurs during the execution of a program, that disrupts the normal flow of the program's instructions.



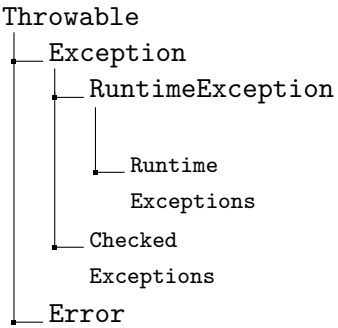


Exception

```
readFile {
    try {
        open the file;
        determine its size;
        allocate that much memory;
        read the file into memory;
        close the file;
    } catch (fileOpenFailed) {
       doSomething;
    } catch (sizeDeterminationFailed) {
        doSomething;
    } catch (memoryAllocationFailed) {
        doSomething;
    } catch (readFailed) {
        doSomething;
    } catch (fileCloseFailed) {
        doSomething;
    }
}
```

Exceptions enable you to write the main flow of your code and to deal with the exceptional cases elsewhere.

Exception



- **Checked Exception**
  exceptional conditions that a well-written application should anticipate and recover from.
  - `java.io.FileNotFoundException`
  - **catch this exception and notify the user of the mistake**
- **Error** (Unchecked Exception)
  External to the application, and the application usually cannot anticipate or recover from.
  - `java.io.IOError`
  - `java.lang.StackOverflowError`
  - **notify the user of the problem or print a stack trace.**
- **Runtime Exception** (Unchecked Exception)
  Internal to the application, and the application usually cannot anticipate or recover from.
  - `java.lang.NullPointerException`
  - `java.lang.ArrayIndexOutOfBoundsException`
  - **debug and eliminate the bug.**

Exception

Difference:

- Checked exceptions have to be caught or thrown
  checked by JavaC at compile time
- No distinction at runtime
  identically treated by the JVM

Advantages of exception handling strategy:

1. Separating error-handling code from "regular" code
2. Propagating errors up the call stack
3. Grouping and differentiating error types

```
method1 {
    try {
        call method2;
    } catch (exception e) {
        doErrorProcessing;
    }
}

method2 throws exception {
    call method3;
}

method3 throws exception {
    call readFile;
}
```

Exception Demo

Throw your own `Exception`.

```
class MyException extends Exception { }

class Main {
    public static void main(String args[]) {
        try {
            throw new MyException();
        }
        catch(Test t) {
            System.out.println("Got the Test Exception");
        }
        finally {
            System.out.println("Inside finally block ");
        }
    }
}
```

Access modifiers

Access Modifiers

Java provides a number of access modifiers to set access levels for *classes*, *variables*, *methods*, and *constructors*.

The four access levels are:

- **Default** (No modifiers are needed): Visible to the package, the default.
- **Private**: Visible to the class only.
- **Public**: Visible to the world.
- **Protected**: Visible to the package and all subclasses.

Private $<$ **Default** $<$ Protected $<$ Public

Access Modifiers

```java
package p1;

public class A {
    protected void display() {
        System.out.println("Com S 228");
    }
}
```

```java
package p2;

import  p1.*;
class B extends A {
    public static void main(String args[]) {
        B obj = new B();
        obj.display();
    }
}
```

Nested Classes

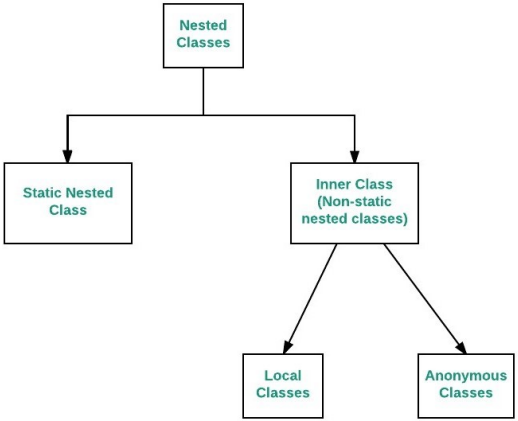
Inner Class and Anonymous Class

Nested classes

Define a class within another class.

A nested class is a member of its enclosing class, which can be declared private, public, protected, or package private.

```
class OuterClass {
    ...
    static class StaticNestedClass {
        ...
    }
    class InnerClass {
        ...
    }
}
```



Why Use Nested Classes?

- **Logically grouping classes that are only used in one place**.
  Nesting such "helper classes" makes their package more streamlined.
- **Increases encapsulation**.
  Consider two top-level classes, A and B, where B needs access to members of A that would otherwise be declared private. A's members can be declared private and B can access them.
- **Lead to more readable and maintainable code**.
  Places the code closer to where it is used.

Anonymous classes

If you need to use a local class only once, use **anonymous class**.

```
interface HelloWorld { public void greet(); }

public void sayHello() {
    class EnglishGreeting implements HelloWorld {
        public void greet() {
            System.out.println("Hello world");
        }
    }
    HelloWorld englishGreeting = new EnglishGreeting();

    HelloWorld frenchGreeting = new HelloWorld() {
        public void greet() {
            System.out.println("Salut tout le monde");
        }
    };
    ...
}
```

Clone and Equals

Object clone method

```
protected Object clone() throws CloneNotSupportedException
```

- Creates and returns a copy of this object. General intent:
  - `x.clone() != x`
  - `x.clone().equals(x)`
  - `x.clone().getClass() == x.getClass()`
- The `Object` class `clone` method makes a shallow copy of the object, but by convention, the object returned by clone method should be **independent** of the object being cloned.

Flawed clone method

```
public class Point implements Cloneable {
    private int x, y;
    ...
    public Point clone() {
        Point copy = new Point(this.x, this.y);
        return copy;
    }
}
```

- What's wrong with the above code?
- What happens if Point is inherited by Point3D

```
// also implements Cloneable and inherits clone()
public class Point3D extends Point {
    private int z;
}
```

Flawed clone method

```
public class Point implements Cloneable {
    private int x, y;
    ...
    public Point clone() {
        Point copy = new Point(this.x, this.y);
        return copy;
    }
}

public class Point3D extends Point {
    private int z;
}
```

The above Point3D class's clone method produces a Point!

- This is undesirable and unexpected behavior.
- The only way to ensure that the clone will have exactly the same type as the original object (even in the presence of inheritance) is to call the clone method from class Object with super.clone()

Proper clone method

```
public class Point implements Cloneable {
    private int x, y;
    ...
    public Point clone() {
        try {
            Point copy = (Point) super.clone();
            return copy;
        } catch (CloneNotSupportedException e) {
            // this will never happen
            return null;
        }
    }
}
```

- To call Object's clone method, you must use try/catch.
- But if you implement Cloneable, the exception will not be thrown

equals() and ==

```
public class Test {
    public static void main(String[] args)
    {
        String s1 = new String("HELLO");
        String s2 = new String("HELLO");
        System.out.println(s1 == s2);
        System.out.println(s1.equals(s2));
    }
}

false
true
```

- == **operator** compares if two primitive values or references are identical.
- `equals()` **method** only compares the values given in s1 and s2.

equals() Demo

```java
class Student {
    String name;
    int id;

    @Override
    public boolean equals(Object obj) {
        if (this == obj) return true;
        if (obj == null) return false;

        // if (!(obj instanceof Student)) return false;
        if (getClass() != obj.getClass())
            return false;

        Student st = (Student) obj;
        return (st.name.equals(this.name) && st.id == this.id);
    }
}
```

Deep copying

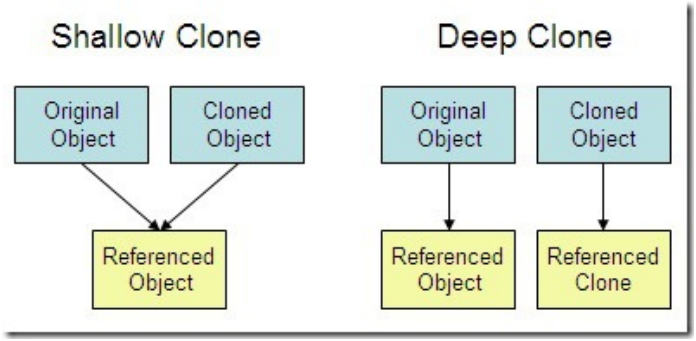
Flawed clone method 2

```
public class BankAccount implements Cloneable {
    private String name;
    private List<String> transactions;
    ...
    public BankAccount clone() {
        try {
            BankAccount copy = (BankAccount) super.clone();
            return copy;
        } catch (CloneNotSupportedException e) {
            return null;   // won't ever happen
        }
    }
}
```

- What's wrong with the above code?

Shallow copy vs deep copy

- **shallow copy**: Duplicates an object without duplicating any other objects to which it refers.
- **deep copy**: Duplicates an object's entire reference graph:
  copies itself and deep copies any other objects to which it refers



Proper clone method 2

```
public class BankAccount implements Cloneable {
   private String name;
   private List<String> transactions;
   ...
   public BankAccount clone() {
      try {              // deep copy
         BankAccount copy = (BankAccount) super.clone();
         copy.transactions = new ArrayList<String>(transactions);
         return copy;
      } catch (CloneNotSupportedException e) {
         return null;   // won't ever happen
      }
   }
}
```

[0] String is immutable type so that we can shallow copy it safely.

Big O notation

Big O Notation

Performance:

- How much time/memory/disk ... is used by the algorithm
- Depends on the machine, compiler, code implementation, etc.

Complexity

- How do the resource requirements of a program/algorithm **scale**?

Big O Notation

```
bool IsFirstElementNull(ArrayList<string> elements)
{
    return elements[0] == null;
}
```

Big O Notation

```
bool ContainsValue(ArrayList<string> elements, string value)
{
    foreach (var element in elements)
    {
        if (element == value) return true;
    }

    return false;
}
```

Big O Notation

```
bool ContainsDuplicates(ArrayList<string> elements)
{
    for (var outer = 0; outer < elements.Count; outer++)
    {
        for (var inner = 0; inner < elements.Count; inner++)
        {
            // Don't compare with self
            if (outer == inner) continue;

            if (elements[outer] == elements[inner]) return true;
        }
    }

    return false;
}
```

Big O Notation

n: scale of the input
$\mathcal{O}(n), \mathcal{O}(n^2), \mathcal{O}(\log n)$: complexity of the algorithm